

MDL-003 — Mental Model

External publication draft

MDL-003 — Mental Model

The interface users see is only the visible expression of the world Mosaic understands.

Purpose

MDL-001 established **why** Mosaic exists.

MDL-002 established **how** design decisions are made.

MDL-003 defines **how Mosaic thinks**.

This specification establishes the conceptual model that every contributor should internalise before designing interfaces, engineering systems or extension APIs.

Unlike a UI framework, the Mental Model describes concepts rather than components.

It explains what exists inside the world of Mosaic independently of how those concepts are eventually rendered.

Relationship to Previous Specifications

Vision

↓

Principles

↓

Mental Model

↓

Interaction Model

↓

Composition Model

↓

Design System

↓

Implementation

Everything below MDL-003 assumes the concepts defined here.

If the Mental Model changes, every downstream specification should be reviewed.

Scope

This specification defines:

- Conceptual architecture

- User mental model
- Platform mental model
- Core concepts
- Relationships between concepts
- Information hierarchy
- World model

This specification intentionally does **not** define:

- Components
- Motion
- Layout
- Tokens
- Materials
- Typography
- Rendering

Those concepts belong to later MDL and MDS specifications.

Core Question

MDL-003 exists to answer one question.

How should contributors think about Mosaic?

Not:

How should they implement it?

Mental Model Statement

Users should never think of Mosaic as:

- a media server
- a dashboard
- a streaming service
- a collection of pages

Instead they should naturally understand it as:

Their personal entertainment world.

Everything within Mosaic should reinforce this understanding.

Primary Concepts

The conceptual hierarchy introduced by MDL-003 is expected to become the foundation for future specifications.

World



Focus



Context



Information



Relationships



Composition



Expressions



Presentation

Each layer answers a different question.

Future chapters define each concept individually.

Expected Outcome

After reading MDL-003 a contributor should naturally understand:

- what exists within Mosaic
- how concepts relate
- what the UI is actually representing
- why composition behaves the way it does
- why plugins contribute information rather than interface

without discussing implementation technologies.

Repository Structure

design/

■■■ mdl/

■■■ MDL-003 Mental Model/

README.md

00-document-control.md

01-what-is-a-mental-model.md

02-the-world.md

03-focus.md

04-context.md

05-information.md

06-relationships.md

07-composition.md

08-expressions.md

09-presentation.md

10-user-vs-system-model.md

11-governance.md

12-adrs.md

13-contributor-guidance.md

glossary.md

references.md

Dependencies

Required reading:

- MDL-001 Vision
- MDL-002 Principles

Downstream specifications:

- MDL-004 Interaction Model
- MDL-005 Composition Model
- MDS-003 Composition Engine
- MDS-011 Extension Design Specification

Review Status

Status

Draft

Owner

Lead Design Systems Architect

Next File

00-document-control.md

Document Control

Document Information

Property	Value
Document ID	MDL-003
Title	Mosaic Design Language — Mental Model
Classification	Internal
Status	Draft
Version	0.1
Owner	Lead Design Systems Architect
Parent Specifications	MDL-001 Vision, MDL-002 Principles
Repository	/design/mdl/MDL-003 Mental Model/

Purpose

MDL-003 establishes the conceptual architecture of Mosaic.

Where MDL-001 explains **why** Mosaic exists and MDL-002 explains **how** design decisions should be made, MDL-003 defines **how the platform understands the world**.

This distinction is critical.

A product's interface is only one expression of its underlying mental model.

If contributors misunderstand the mental model, every implementation built upon it becomes inconsistent regardless of visual quality.

Authority

MDL-003 governs the conceptual architecture of Mosaic.

Its authority extends to:

- Product Design
- User Experience
- Information Architecture
- Composition
- Extension Architecture
- GraphQL UI
- Runtime Composition
- Design Systems

MDL-003 intentionally does **not** prescribe:

- visual appearance
- implementation technologies
- rendering techniques
- framework-specific behaviour

Those concerns belong to MDS.

Relationship To MDL

The Mosaic Design Language intentionally separates philosophy from implementation.

```
[mermaid]
flowchart TD
```

```
Vision["MDL-001 Vision"]
```

```
Principles["MDL-002 Principles"]
```

```
Mental["MDL-003 Mental Model"]
```

```
Interaction["MDL-004 Interaction Model"]
```

```
Composition["MDL-005 Composition Model"]
```

```
DesignSystem["MDS Specifications"]
```

```
Implementation["Product"]
```

```
Vision --> Principles
Principles --> Mental
Mental --> Interaction
Interaction --> Composition
Composition --> DesignSystem
DesignSystem --> Implementation
```

MDL-003 acts as the bridge between philosophy and behaviour.

Every interaction described by later specifications should be understandable through the concepts introduced here.

Design Intent

The purpose of a mental model is not to describe software.

It is to describe reality as Mosaic understands it.

Users should never need to understand:

- databases
- APIs
- GraphQL
- plugins
- rendering engines

Instead they should instinctively understand concepts such as:

- World
- Focus
- Context
- Relationships

Everything else exists to support those concepts.

Why Mental Models Matter

People build mental models regardless of whether designers intentionally create one.

Poor products force users to invent inconsistent explanations.

Good products provide a simple conceptual model that naturally explains every interaction.

The objective of MDL-003 is therefore:

One product.

One conceptual model.

Infinite experiences.

Specification Objectives

Upon completing MDL-003, contributors should understand:

- what fundamentally exists inside Mosaic
- how information is organised
- why the interface behaves as it does
- how extensions integrate
- why composition exists
- how future systems should evolve

without requiring implementation examples.

Conceptual Stability

The concepts introduced by MDL-003 are expected to remain stable significantly longer than implementation.

Expected lifespan.

Artefact	Expected Lifetime
Implementation	Months
Components	Months
Patterns	Years

Artefact	Expected Lifetime
Mental Model	Decades

Changing the Mental Model should therefore be considered a major architectural event.

Reader Expectations

Before continuing, contributors should already understand:

- MDL-001 Vision
- MDL-002 Principles

MDL-003 intentionally assumes familiarity with those documents.

It does not repeat them.

Instead, it builds upon them.

Success Criteria

MDL-003 succeeds when:

- contributors naturally describe Mosaic using the same concepts
- plugins integrate without introducing new conceptual models
- engineering systems reinforce rather than redefine product behaviour
- users rarely need explanations for how the product works

When contributors begin independently using terms such as:

- World
- Focus
- Context
- Composition

without conscious effort...

the Mental Model has become part of the culture of the project.

Review Status

Status

Draft

Dependencies

- MDL-001 Vision
- MDL-002 Principles

Supersedes

None.

Next File

01-what-is-a-mental-model.md

What Is A Mental Model?

Purpose

Before defining the concepts that exist within Mosaic, contributors must first understand what a mental model is and why it matters.

The purpose of this chapter is to establish a shared understanding of the role the Mental Model plays within the Mosaic Design Language.

Without a common conceptual model, contributors naturally invent their own.

Those individual interpretations eventually become inconsistent interfaces, inconsistent APIs and inconsistent experiences.

The Mental Model exists to prevent that fragmentation.

Definition

Within the Mosaic Design Language, a **Mental Model** is:

The conceptual explanation of how Mosaic understands the user's world.

It is intentionally independent from:

- user interface
- implementation
- programming language
- framework
- database
- network protocol

The Mental Model defines concepts.

Implementation expresses those concepts.

Why Mental Models Exist

Every user forms an internal explanation of how software works.

This explanation is rarely technically accurate.

It does not need to be.

It simply needs to be:

- consistent
- predictable
- understandable

People successfully use cars without understanding combustion engines.

People successfully use electricity without understanding electrical engineering.

Likewise, Mosaic users should successfully use the platform without understanding GraphQL, plugins or runtime composition.

They should understand only the concepts required to accomplish their goals.

Mental models are a recognised UX concept describing the internal explanation users construct to predict how a system behaves. Good products align the system model with the user's mental model to reduce confusion and cognitive effort. (Norman, *The Design of Everyday Things*.)

System Model vs Mental Model

Every software product contains two models.

System Model

The technical reality.

Examples include:

- GraphQL
- SQLite
- DuckDB
- WebSockets
- Plugins
- Composition Engine

The system model belongs to engineers.

Mental Model

The conceptual reality.

Examples include:

- World
- Focus
- Context
- Relationships
- Progress
- Companion

The mental model belongs to users.

These models intentionally differ.

The purpose of design is to translate one into the other.

A Bad Mental Model

Imagine a media application whose navigation is organised around:

Media

↓

Series

↓

Season

↓

Episode

↓

Playback

Internally this may accurately describe the database.

From the user's perspective it introduces unnecessary concepts.

The user is forced to understand software architecture rather than entertainment.

A Better Mental Model

Mosaic instead proposes:

My World

↓

My Current Focus

↓

What Helps Me Right Now

This is significantly closer to how people naturally describe entertainment.

Notice that:

- databases disappear
- APIs disappear
- plugins disappear
- implementation disappears

Only meaningful concepts remain.

Mental Models Are Not Navigation

A common mistake is assuming that the Mental Model describes interface layout.

It does not.

Navigation is one possible expression of the Mental Model.

So are:

- composition
- search
- recommendations
- playback
- administration
- notifications

Every part of Mosaic should reinforce the same conceptual understanding.

Mental Models Are Stable

Implementation changes frequently.

Mental models should not.

Changing the Mental Model requires contributors to relearn how the product works.

This is one of the most expensive forms of design change.

Consequently:

- components may evolve
- composition may evolve
- rendering may evolve

The conceptual model should evolve only when significant evidence demonstrates that it no longer reflects the intended experience.

The Cost Of Multiple Mental Models

Products often become inconsistent because different teams unknowingly build different conceptual models.

Examples include:

Search Team:

Mosaic is a search application.

Playback Team:

Mosaic is a media player.

Library Team:

Mosaic is a database.

Plugin Team:

Mosaic is an extension host.

Each statement is partially true.

None describes the complete product.

MDL-003 exists so every team begins from the same understanding.

The Mosaic Mental Model

The conceptual model of Mosaic is intentionally human rather than technical.

The user does not experience:

Database

↓

API

↓

Renderer

They experience:

My World

↓

What I'm Enjoying

↓

What Helps Me Continue

Everything implemented by Mosaic should reinforce this understanding.

The Test

A contributor should be capable of explaining Mosaic to a non-technical family member without mentioning:

- GraphQL
- APIs
- plugins
- databases
- servers
- rendering

If that explanation remains accurate...

The Mental Model is working.

Relationship To The Remaining Specification

The remaining chapters progressively define the concepts that make up the Mosaic Mental Model.

Beginning with:

World

↓

Focus

↓

Context

↓

Information

↓

Relationships

↓

Composition

↓

Expressions

↓

Presentation

Every later chapter builds upon the previous one.

No concept should be understood in isolation.

Summary

The Mental Model is the conceptual architecture of Mosaic.

It describes:

- what exists
- why it exists
- how those concepts relate

without describing how any of them are implemented.

Every interface, API and future extension should reinforce this same conceptual understanding.

Review Status

Status

Draft

Next File

02-the-world.md

The World

Purpose

The World is the highest-level concept within the Mosaic Mental Model.

Everything else described within MDL-003 exists inside a World.

Understanding this concept is fundamental because it changes how contributors think about the product.

Traditional media applications organise themselves around:

- libraries
- pages
- folders
- media collections

Mosaic intentionally does not.

Instead, Mosaic organises itself around **the user's entertainment world**.

Definition

Within MDL, a **World** is defined as:

The complete representation of a person's entertainment life at a given point in time.

A World is not:

- a page
- a library
- a dashboard
- a profile

Those are merely ways a World may be expressed.

The World is the conceptual environment within which every entertainment experience exists.

Why "World"?

People rarely think about entertainment as isolated pieces of media.

Instead they naturally describe worlds.

Examples include:

"I'm watching loads of medical dramas."

"I've been reading Brandon Sanderson recently."

"I've gone back into Marvel."

"I'm catching up on this anime season."

These statements describe temporary entertainment worlds.

Not files.

Not libraries.

Not folders.

The Mental Model intentionally reflects how people naturally describe their own entertainment.

A World Is Personal

Every user possesses a different World.

No two Worlds are identical.

A World reflects:

- interests
- current focus
- progress
- relationships
- collections
- history
- available entertainment

The World should therefore feel personal rather than generic.

When users open Mosaic they should immediately recognise:

This is my world.

Not:

"This is the application's homepage."

The World Is Continuous

The World does not reset when users change media.

Watching:

Frieren

Reading:

The Hobbit

Listening:

Hans Zimmer

These are not separate applications.

They are different areas within the same World.

This continuity allows Mosaic to remove unnecessary mental transitions between media types.

The World Evolves

A World is not static.

It continuously evolves as:

- progress changes
- episodes release
- books are completed
- albums are discovered
- plugins contribute information
- relationships become available

Importantly...

The World evolves.

It is not rebuilt.

This distinction becomes critical within later specifications covering interaction and composition.

World Boundaries

A World intentionally contains everything required to understand a person's entertainment.

It includes:

- Domains
- Focus
- Context
- Information
- Relationships
- Collections
- History

It intentionally excludes:

- interface
- navigation
- layout
- rendering
- components

These belong to the presentation layer.

Domains

A World contains one or more Domains.

Examples include:

World

■■■ Anime

■■■ Television

■■■ Books

■■■ Movies

■■■ Music

Domains organise entertainment.

They do not separate experiences.

Users remain inside the same World while moving between Domains.

Multiple Worlds

Future Mosaic capabilities may introduce multiple Worlds.

Examples include:

- Personal World
- Family World
- Shared Household World
- Child World
- Temporary Watch Party World

The concept remains unchanged.

Each World simply represents a different entertainment environment.

Designing around "World" rather than "Home" allows future capabilities without redefining the Mental Model.

Entering A World

Opening Mosaic should never feel like opening software.

Instead it should feel like returning to an existing place.

The first responsibility of the interface is therefore to answer one question.

What does my entertainment world look like right now?

Only after establishing the World should Mosaic begin presenting:

- focus
- context
- relationships
- recommendations
- progression

Everything derives from the World.

Good Examples

Example 01

The user opens Mosaic.

Immediately visible:

- Continue Watching
- Current book
- New album from favourite artist
- Tomorrow's anime episode

The interface communicates:

"This is your world."

Example 02

The user returns after six months.

Instead of overwhelming them with everything that changed...

The World gently reflects what has changed within the entertainment they were previously enjoying.

Only when the user's Focus changes does another area of the World become dominant.

The World remains continuous.

Only emphasis changes.

Anti-patterns

The following behaviours conflict with the World model.

Dashboard Thinking

Displaying every possible metric simultaneously.

Users see software.

Not their entertainment.

Application Switching

Treating books, music and television as independent products.

The World fragments.

Continuity disappears.

Homepage Thinking

Assuming there is one static "home".

There is not.

The World is always changing.

The current composition merely expresses it.

Conceptual Model

[mermaid]
flowchart TD

World

--> Domains

Domains --> Focus

Focus --> Context

Context --> Information

Information --> Relationships

Relationships --> Composition

Everything inside Mosaic ultimately exists within a World.

Nothing should bypass this hierarchy.

Design Consequences

Treating the World as the primary concept has significant consequences.

The interface becomes:

- continuous
- contextual
- adaptive
- personal

Navigation becomes:

- movement within a World

rather than

- movement between pages.

This conceptual shift forms the foundation for every subsequent chapter within MDL-003.

Summary

The World is the highest-level concept within Mosaic.

It represents the user's complete entertainment environment.

Everything else exists inside it.

Users should never feel they are moving between applications.

They should feel they are exploring different parts of the same World.

Review Status

Status

Draft

Next File

03-focus.md

Focus

Purpose

If the **World** defines *where* the user currently exists, **Focus** defines *what currently matters*.

Focus is the central organising concept of the Mosaic experience.

Every composition.

Every recommendation.

Every interaction.

Every transition.

Every adaptive behaviour ultimately exists to support the user's current Focus.

Without Focus, a World becomes an undifferentiated collection of information.

With Focus, Mosaic understands what deserves the user's attention.

Definition

Within MDL, **Focus** is defined as:

The entertainment experience currently occupying the user's attention.

Focus is intentionally singular.

Although users may have many interests, they generally concentrate on one primary activity at any given moment.

Examples include:

- Watching *Frieren*
- Reading *The Hobbit*
- Listening to *Hans Zimmer Live*
- Browsing Studio Ghibli films

Focus is not:

- favourite media
- highest-rated media
- recently added media

Focus always describes **the present**.

Why Focus Exists

Traditional media software asks:

"What media should we display?"

Mosaic instead asks:

"What currently deserves the user's attention?"

This is a subtle but fundamental distinction.

Media is static.

Attention is dynamic.

The interface should therefore organise itself around attention rather than inventory.

One World, One Focus

Every World contains many possibilities.

Only one should normally be considered the primary Focus.

World

■■■ Anime

■■■ Books

■■■ Music

■■■ Movies

■■■ Television

↓

Current Focus

↓

Frieren

The interface should communicate this clearly.

Everything else becomes supporting context.

Focus Is Dynamic

Focus changes naturally.

Examples include:

Morning

↓

Podcast

↓

Afternoon

↓

Book

↓

Evening

↓

Television

The World remains stable.

Only Focus changes.

Future specifications define how compositions adapt to these changes.

Focus Is Not Selection

Selecting an item does not automatically make it the user's Focus.

Example.

A user briefly opens:

Breaking Bad

to check the cast.

Their current entertainment journey may still be:

Frieren

Focus should represent genuine intent.

Not transient interaction.

This distinction allows Mosaic to remain contextually stable rather than reacting to every click.

Determining Focus

Future implementation may derive Focus from many signals.

Examples include:

- active playback
- active reading
- current listening session
- recent interaction
- explicit user intent

MDL intentionally avoids prescribing the implementation.

Regardless of implementation, Focus should remain predictable from the user's perspective.

Users should rarely be surprised by what Mosaic considers important.

Focus Creates Hierarchy

Once Focus is established, hierarchy naturally follows.

Focus

↓

Supporting Information

↓

Related Information

↓

Background Information

This ordering allows every composition to answer the user's most immediate questions first.

The interface no longer needs arbitrary "featured" sections.

Importance emerges from Focus.

Examples

Example 01

Current Focus

Frieren

Supporting information:

- Watch progress
- Next episode
- Manga continuation
- Characters
- Soundtrack

The composition naturally organises around the current Focus.

Example 02

Current Focus

Project Hail Mary

Supporting information:

- Reading progress
- Chapter
- Audiobook
- Andy Weir

- Related novels

The same systems apply.

Only the Focus changes.

Losing Focus

Sometimes users intentionally leave their current Focus.

Examples include:

- searching for new media
- browsing collections
- changing domains

During these moments Mosaic should gracefully reduce emphasis on the previous Focus.

It should not immediately discard it.

Focus transitions should preserve continuity.

Future specifications describe these transitions in detail.

Anti-patterns

The following behaviours violate the Focus model.

Multiple Equal Priorities

Displaying several competing hero experiences simultaneously.

Users lose understanding of what currently matters.

Algorithmic Focus

Allowing popularity metrics to determine Focus.

Focus belongs to the user.

Not the platform.

Interface Focus

Making navigation or software controls more visually important than the user's entertainment.

The interface has become the Focus.

This is explicitly rejected by MDL.

Relationship To Context

Focus and Context are closely related.

They are not identical.

Focus

↓

What currently matters.

Context

↓

Why it currently matters.

Example.

Focus

Frieren

↓

Context

Watching Episode 14

Without Focus there is no subject.

Without Context there is no explanation.

Together they allow Mosaic to understand the user's current experience.

Conceptual Model

[mermaid]
flowchart TD

```
World
World --> Focus
Focus --> Context
Context --> Information
Information --> Relationships
Relationships --> Composition
```

Focus acts as the bridge between the user's World and the information Mosaic chooses to emphasise.

Design Consequences

Treating Focus as a first-class concept produces several important behaviours.

The interface naturally develops:

- one visual centre
- one current journey
- one dominant hierarchy

The user no longer needs to ask:

"What should I look at first?"

The composition already answers that question.

Summary

Focus represents the entertainment experience currently deserving the user's attention.

It is the organising centre of every Mosaic composition.

The World remains constant.

Focus changes.

Everything else adapts naturally around it.

Review Status

Status

Draft

Next File

04-context.md

Context

Purpose

If **Focus** answers:

"What currently matters?"

then **Context** answers:

"Why does it matter right now?"

Context transforms information into understanding.

Without Context, Mosaic becomes a sophisticated catalogue.

With Context, Mosaic becomes a companion.

Context allows the platform to understand not only **what** the user is interacting with, but **how** that interaction should influence the rest of the experience.

Definition

Within MDL, **Context** is defined as:

The temporary circumstances surrounding the user's current Focus that determine what information is relevant at this moment.

Context exists to answer:

- Why is this important?
- What should happen next?
- What information belongs here?
- What should be hidden?

Context is temporary.

It changes continuously.

Why Context Exists

Entertainment rarely exists in isolation.

Watching an episode.

Reading a chapter.

Listening to an album.

Browsing a franchise.

Each activity creates a different set of questions.

Watching an episode naturally creates questions such as:

- Which episode is next?
- When does it release?
- How far through am I?
- Is there a manga continuation?

Reading a novel creates different questions:

- Which chapter am I on?
- How much remains?
- What other books has the author written?

The content has changed.

The questions have changed.

Therefore the Context has changed.

Focus And Context

Focus and Context are intentionally separate concepts.

Focus

↓

The subject.

Context

↓

The circumstances.

Example.

Focus

Frieren

Possible Contexts:

- Watching Episode 14
- Browsing Characters
- Looking at Reviews
- Reading Manga
- Searching Similar Anime

The Focus remains identical.

The Context changes.

Consequently the interface should adapt.

Context Is Temporary

Context should never permanently redefine the user's World.

Example.

Current Context

Searching

Searching is not a permanent state.

Once the search is complete, the previous Context should naturally return.

Likewise:

Browsing Settings

does not replace:

Watching Frieren

Administration is temporary.

Entertainment remains the user's World.

Context Is Layered

Context may exist at several levels simultaneously.

Example.

World

↓

Anime

↓

Frieren

↓

Episode 14

↓

Watching

Each layer adds understanding.

The interface should respond primarily to the most immediate Context while preserving awareness of the surrounding layers.

Context Defines Relevance

Context determines what information deserves emphasis.

Without Context:

Every piece of information appears equally valuable.

With Context:

Information naturally forms hierarchy.

Example.

Watching:

Episode 14

Relevant:

- Next Episode
- Remaining Runtime
- Playback Controls
- Watch Progress

Less Relevant:

- Complete Cast List
- Studio History
- Merchandise

The information still exists.

It simply no longer deserves primary attention.

Context Is Not History

History describes what happened.

Context describes what is happening.

Example.

History:

Finished Episode 13

Context:

Watching Episode 14

History informs Context.

It does not replace it.

Future systems may use history to infer Context, but the two concepts remain distinct.

Context Is Not Prediction

Prediction attempts to answer:

What will the user want next?

Context answers:

What does the user need right now?

This distinction reflects **Principle 01 — Context Before Prediction**.

Prediction may become useful later.

Context always comes first.

Good Examples

Example 01

Current Context

Watching Episode

Interface emphasises:

- playback
- remaining runtime
- subtitles
- audio
- next episode

Everything supports the current activity.

Example 02

Current Context

Reading

Interface emphasises:

- chapter
- reading progress
- bookmarks
- notes

Playback controls are irrelevant.

They quietly disappear.

Example 03

Current Context

Browsing Franchise

Interface emphasises:

- timeline

- relationships
- adaptations
- chronology

The user is exploring.

The interface should help exploration.

Anti-patterns

The following behaviours violate the Context model.

Static Interfaces

Every screen displays identical information regardless of what the user is doing.

Context has been ignored.

Context Switching Without Explanation

The interface suddenly changes emphasis without an understandable reason.

The user loses trust.

Predictive Overrides

The interface ignores the current Context because an algorithm believes something else should be important.

Current understanding has been replaced by speculation.

Context Drives Composition

Context does not directly create interface.

Instead it influences composition.

World

↓

Focus

↓

Context

↓

Composition

The same information may appear very differently depending upon Context.

Future specifications define these transformations.

Plugin Behaviour

Extensions should never attempt to own Context.

Instead they contribute information relevant to the existing Context.

Example.

Anime Extension

Episode 15

↓

Tomorrow

The platform determines whether that information is:

- Hero
- Timeline
- Badge
- Notification
- Metadata

The plugin contributes knowledge.

The platform contributes understanding.

Design Consequences

Treating Context as a first-class concept produces several important behaviours.

The interface becomes:

- adaptive
- predictable
- relevant
- calmer

Users begin to trust that the information they see has been chosen because it is useful now.

Not because it was simply available.

Conceptual Model

```
[mermaid]
flowchart TD
    World --> Focus
    Focus --> Context
    Context --> RelevantInformation["Relevant Information"]
    RelevantInformation --> Composition
    Composition --> Presentation
```

Context determines relevance.

Relevance determines composition.

Composition determines presentation.

Summary

Context explains the user's current situation.

It determines what deserves attention.

It allows the same World and the same Focus to produce many different experiences without fragmenting the user's mental model.

Without Context, Mosaic cannot become a companion.

It can only become another catalogue.

Review Status

Status

Draft

Next File

05-information.md

Information

Purpose

Information is the first concept within the Mosaic Mental Model that exists independently from the user interface.

Everything before this chapter has described **how Mosaic understands the user's entertainment world**.

This chapter begins describing **what actually exists inside that world**.

The distinction is intentional.

Mosaic should fundamentally understand **information**, not interface.

The interface is simply one possible way of communicating that information.

This concept becomes the foundation for the Composition Engine, Extension SDK and future runtime systems.

Definition

Within MDL, **Information** is defined as:

A single, meaningful piece of knowledge describing the user's entertainment world.

Information is atomic.

It cannot be meaningfully divided into smaller concepts without losing semantic value.

Examples include:

- Episode 14 releases tomorrow.
- Current watch progress is 68%.
- Chapter 12 has been completed.
- Hans Zimmer composed this soundtrack.
- This film is based on a novel.
- This actor also appears in another series.

These statements remain meaningful regardless of how they are eventually presented.

Why Information Exists

Traditional UI systems frequently begin with interface.

Examples include:

Timeline Widget

Recommendation Card

Progress Bar

Metadata Panel

These are interface concepts.

They describe presentation.

Mosaic intentionally begins one layer earlier.

Instead it asks:

What do we actually know?

Examples.

Episode Release

Progress

Relationship

Author

Runtime

Genre

The platform should understand these concepts before deciding how they should appear.

Information Before Interface

This distinction is fundamental.

The following statement should be considered a core architectural assumption of Mosaic.

Plugins contribute information.

Mosaic contributes interface.

Future engineering systems should therefore avoid coupling knowledge to presentation.

Information Is Permanent

Interfaces evolve.

Information rarely does.

Example.

Episode Release

↓

Tomorrow

Today this information may appear as:

- Timeline
- Hero
- Notification

Tomorrow it may appear as:

- Calendar
- Voice Assistant
- Smart TV Overlay

The information remains identical.

Only its expression changes.

This separation allows the platform to evolve without redefining its underlying knowledge.

Characteristics Of Information

Every Information object should satisfy the following properties.

Meaningful

It communicates something useful.

Independent

It exists independently from presentation.

Reusable

Many different compositions may consume the same information.

Relational

Information may connect to other information.

Observable

Changes may trigger updates elsewhere within the system.

Information Categories

Although future specifications will formalise the complete Information Model, information generally belongs to one of the following categories.

Progress

Examples:

- watch progress
- reading progress
- listening progress

Time

Examples:

- release date
- runtime
- publication date
- countdown

Identity

Examples:

- title
- author
- studio
- artist
- cast

Relationship

Examples:

- sequel
- adaptation
- soundtrack
- franchise
- shared actor

Status

Examples:

- airing
- completed
- ongoing
- paused

Availability

Examples:

- available locally
- streaming

- downloaded
- offline

These categories intentionally describe meaning.

Not presentation.

Information Is Universal

The same conceptual information should exist regardless of media type.

Example.

Progress.

Television.

Episode 12

68%

Books.

Chapter 9

68%

Music.

Album

68%

The Progress concept remains identical.

Only the domain changes.

This allows Mosaic to build reusable systems rather than media-specific implementations.

Good Examples

Example 01

Anime plugin.

Provides:

Episode Release

↓

Tomorrow

The plugin does **not** decide how that information appears.

Example 02

Book plugin.

Provides:

Reading Progress

↓

Chapter 12

↓

68%

Again...

Information.

Not interface.

Example 03

Movie metadata.

Provides:

Actor

↓

Appears In

↓

Interstellar

Relationship.

Not navigation.

Anti-patterns

The following violate the Information model.

Returning Interface

Timeline Widget

Wrong.

Returning HTML

<div>

...

Wrong.

Returning Layout

Left Panel

↓

Metadata

↓

Card

Wrong.

The plugin has started designing the interface.

That responsibility belongs to Mosaic.

Information And Relationships

Information rarely exists alone.

Example.

Episode 15

↓

Tomorrow

↓

Frieren

These concepts become useful because they relate to one another.

The next chapter formalises these relationships.

Information provides facts.

Relationships provide meaning.

Information Flow

The conceptual flow introduced by MDL is therefore:

```
[mermaid]
flowchart TD
```

```
World
World --> Focus
Focus --> Context
Context --> Information
Information --> Relationships
Relationships --> Composition
Composition --> Expressions
Expressions --> Presentation
```

Notice that presentation appears almost at the end of the process.

This ordering is intentional.

Engineering Implications

Although implementation belongs within MDS specifications, this Mental Model establishes several architectural expectations.

Future systems should allow:

- multiple consumers of the same information
- multiple presentations of the same information
- multiple devices sharing the same information
- plugins contributing information without UI knowledge

This architectural flexibility is considered one of the primary long-term advantages of the Mosaic platform.

Summary

Information is the first concept within MDL that exists independently from interface.

The platform should understand:

What is true

before deciding:

How it should be shown.

Every future Mosaic experience should therefore begin with Information rather than presentation.

Review Status

Status

Draft

Next File

06-relationships.md

Relationships

Purpose

Information, by itself, is useful.

Relationships transform information into understanding.

This chapter introduces one of the most important concepts within the Mosaic Mental Model.

Without relationships, Mosaic becomes a searchable database.

With relationships, Mosaic becomes an entertainment companion.

Relationships explain **why** pieces of information matter together.

Definition

Within MDL, a **Relationship** is defined as:

A meaningful connection between two or more pieces of information.

Relationships create context.

Relationships create understanding.

Relationships create discovery.

Information provides facts.

Relationships provide meaning.

Why Relationships Exist

People rarely experience entertainment as isolated media.

Instead they naturally connect experiences.

Examples include:

"That actor was also in..."

"This anime was adapted from..."

"The composer also wrote..."

"The next book continues..."

These connections are not interface features.

They are relationships that already exist.

Mosaic's responsibility is to reveal them naturally.

Relationships Before Navigation

Traditional applications encourage navigation.

Movie



Actor Page



Filmography



Another Movie

Every step requires navigation.

Mosaic instead understands the relationship directly.

Movie



Actor



Related Works

The interface simply reveals that relationship.

The user explores naturally rather than navigating mechanically.

Types Of Relationships

Relationships exist at many levels.

Sequential

Describes progression.

Examples include:

- next episode
- previous episode
- next chapter
- sequel
- chronological order

Structural

Describes ownership or composition.

Examples include:

- season belongs to series
- episode belongs to season
- chapter belongs to book
- track belongs to album

Creative

Describes people and production.

Examples include:

- actor
- director
- composer
- author
- illustrator
- studio

Narrative

Describes fictional connections.

Examples include:

- spin-off
- adaptation
- prequel
- alternate timeline
- shared universe

Personal

Describes relationships to the user.

Examples include:

- currently watching
- completed
- bookmarked
- favourite
- continue reading

These relationships are unique to each World.

Relationships Are First-Class Concepts

A common mistake is treating relationships as metadata.

Within MDL they are significantly more important.

Metadata describes objects.

Relationships describe worlds.

Consider:

Frieren

↓

Fantasy

Metadata.

Now consider:

Frieren

↓

Adapted From

↓

Manga

Relationship.

The second naturally leads somewhere.

The first merely categorises.

Mosaic should favour meaningful relationships over passive categorisation whenever practical.

Relationship Density

Not every relationship deserves equal emphasis.

Some relationships are fundamental.

Others are incidental.

Example.

Frieren

↓

Episode 15

Strong.

Frieren

↓

Composer

Medium.

Frieren

↓

Won Award

↓

2023

Weak.

The strength of a relationship should influence future composition systems.

Not every relationship deserves equal visual weight.

Relationships Are Directional

Relationships intentionally possess direction.

Example.

Book

↓

Adapted Into

↓

Film

is not necessarily identical to

Film

↓

Based On

↓

Book

Both describe the same connection.

Each answers a different user question.

Future engineering systems should preserve relationship direction where practical.

Relationships Build Worlds

A World is not simply a collection of information.

It is a network.

[mermaid]
graph TD

World

--> Frieren

Frieren --> Episode15

Frieren --> Manga

Frieren --> Composer

Frieren --> Studio

Frieren --> Characters

As relationships increase...

The World becomes richer.

Not because more information exists.

Because understanding increases.

Good Examples

Example 01

Current Focus

Dune

Relationships.

- sequel
- author
- audiobook
- film
- soundtrack

The interface naturally supports exploration.

Example 02

Current Focus

Hans Zimmer

Relationships.

- Interstellar
- Dune
- Gladiator
- Live Performances

No recommendation engine required.

The relationships already exist.

Example 03

Current Focus

Breaking Bad

Relationships.

- Better Call Saul
- El Camino
- Vince Gilligan
- Bryan Cranston

Again...

The experience expands naturally.

Anti-patterns

The following behaviours weaken the relationship model.

Flat Lists

Movie

Movie

Movie

Movie

Nothing explains why these belong together.

Artificial Relationships

Trending

↓

Because Popular

Popularity is not a meaningful relationship.

It is a ranking.

Hidden Relationships

Relationships exist but remain inaccessible behind unnecessary navigation.

Understanding becomes fragmented.

Relationships And Composition

Relationships do not directly determine interface.

They influence composition.

Information

↓

Relationships

↓

Importance

↓

Composition

This distinction is critical.

Relationships create meaning.

Composition communicates meaning.

Presentation renders composition.

Each layer possesses one responsibility.

Plugins

Plugins should contribute relationships.

Not interface.

Example.

Anime Extension.

Episode

↓

Next Episode

Book Extension.

Book

↓

Audiobook

Music Extension.

Artist

↓

Concert

The platform remains responsible for deciding:

- emphasis
- hierarchy
- timing
- interaction

Relationships enrich the World.

The platform decides how users experience them.

Relationship Graph

Long-term, Mosaic should increasingly understand entertainment as a graph rather than a hierarchy.

Traditional hierarchy.

Series

↓

Season

↓

Episode

Relationship graph.

[mermaid]
graph LR

Episode

--- Series

Episode

--- Manga

Series

--- Studio

Series

--- Composer

Series

--- Characters

Characters

--- VoiceActors

This richer model enables future experiences without requiring additional interface concepts.

Summary

Relationships transform information into understanding.

Without relationships...

Mosaic organises media.

With relationships...

Mosaic understands entertainment.

This distinction is one of the defining characteristics of the Mosaic Mental Model.

Review Status

Status

Draft

Next File

07-composition.md

Composition

Purpose

If Information defines **what** Mosaic knows, and Relationships define **how those pieces of information connect**, Composition defines:

What deserves the user's attention right now.

Composition is not a layout engine.

It is a decision-making engine.

Its responsibility is to transform understanding into experience.

Composition determines:

- emphasis
- hierarchy
- grouping
- priority
- visual rhythm

It intentionally does **not** determine colours, typography or components.

Those belong to the Mosaic Design System.

Definition

Within MDL, **Composition** is defined as:

The intentional organisation of information that communicates the user's current World with the least possible cognitive effort.

Composition exists before interface.

The interface merely renders the composition.

Why Composition Exists

Without composition every piece of information competes equally.

Example.

Continue Watching

↓

Upcoming Episode

↓

Author

↓

Runtime

↓

Genres

↓

Cast

↓

Reviews

↓

Studio

Everything is technically useful.

Nothing is clearly important.

Composition exists to answer one question.

"What should the user notice first?"

Composition Is Not Layout

This distinction is fundamental.

Layout answers:

Where should things be placed?

Composition answers:

Why should something deserve attention?

Layout is geometric.

Composition is conceptual.

Many different layouts may successfully express the same composition.

Composition Is Temporary

A composition exists only for the current moment.

As Focus changes...

Composition changes.

As Context changes...

Composition changes.

As Relationships become more relevant...

Composition changes.

The World remains stable.

Composition evolves continuously.

The Responsibility Of Composition

Composition determines:

- emphasis
- hierarchy
- grouping
- density
- breathing space
- progression

Composition intentionally does **not** determine:

- spacing values
- colours
- shadows
- typography
- motion curves

Those are implementation concerns.

Inputs

Composition consumes concepts.

Not interface.

Its inputs include:

World

↓

Focus

↓

Context

↓

Information

↓

Relationships

These inputs describe understanding.

Not presentation.

Outputs

Composition produces:

- priority
- importance
- grouping
- emphasis
- structure

These outputs are later transformed into interface by the Design System.

Future specifications introduce **Expressions** as the bridge between composition and presentation.

The Goal Of Composition

The primary objective of composition is:

Reduce interpretation.

A user should not need to ask:

- What matters?
- Where do I begin?
- Why is this here?
- What should I do next?

A good composition answers those questions before they are consciously asked.

One Composition

Every experience should possess one dominant composition.

Example.

Watching

↓

Playback

↓

Progress

↓

Next Episode

↓

Related Information

Not:

Playback

Recommendations

Trending

Downloads

News

Plugins

Friends

Settings

Statistics

Multiple competing centres of attention create unnecessary cognitive work.

Composition Evolves

Composition should never appear to rebuild itself.

Instead it should evolve.

Example.

Episode Complete

↓

Progress Updates

↓

Next Episode Gains Priority

↓

Timeline Expands

↓

Playback Controls Reduce

Nothing has appeared arbitrarily.

The composition has simply reorganised itself.

Future specifications formalise this behaviour through the Interaction Model.

Composition Is Intentional

Every element within a composition should justify its existence.

If removing something produces no noticeable reduction in understanding...

It probably did not belong there.

Composition is therefore an editorial process.

Not an additive one.

Density

Composition determines information density.

Some situations naturally require:

Sparse

↓

High Focus

↓

Large Hierarchy

Others require:

Dense

↓

Exploration

↓

Rich Relationships

Neither is inherently correct.

Density should emerge from Context rather than arbitrary layout rules.

Breathing Space

Whitespace is not unused interface.

Whitespace communicates:

- hierarchy
- confidence
- rhythm
- separation

Composition should intentionally leave room for change.

Crowding information simply because space exists weakens understanding.

Grouping

Information should be grouped according to meaning.

Not implementation.

Good grouping.

Current Episode

↓

Progress

↓

Next Episode

Poor grouping.

Progress

↓

Plugin

↓

Cast

↓

Settings

↓

Recommendations

Related concepts should remain visually close because they are conceptually close.

Priority

Every piece of information possesses priority.

Priority is contextual.

Example.

Runtime

During playback.

Low priority.

Before playback.

Higher priority.

Context changes priority.

Priority changes composition.

Composition changes presentation.

Progressive Disclosure

Composition should reveal information progressively.

Users should never be overwhelmed simply because more information exists.

Example.

Focus

↓

Essential Information

↓

Supporting Information

↓

Exploration

Information should appear when it becomes useful.

Not merely because it exists.

Composition Is Device Independent

Composition is conceptual.

Desktop.

Television.

Mobile.

Tablet.

All may express the same composition differently.

The hierarchy remains identical.

Only presentation changes.

Good Examples

Watching

Focus

↓

Playback

↓

Progress

↓

Next Episode

↓

Related Works

Simple.

Clear.

Predictable.

Reading

Focus



Current Chapter



Reading Progress



Bookmarks



Author

Again...

The composition answers the user's immediate questions.

Anti-patterns

Dashboard Thinking

Attempting to display every available metric simultaneously.

Equal Weight

Every section appears equally important.

Nothing naturally attracts attention.

Random Layout

Information arranged according to implementation convenience rather than conceptual relationships.

Static Composition

Every screen contains identical hierarchy regardless of changing Context.

Composition Pipeline

[mermaid]
flowchart TD

```
World
World --> Focus
Focus --> Context
Context --> Information
Information --> Relationships
Relationships --> Composition
Composition --> Expressions
Expressions --> Presentation
```

Composition is the final conceptual step before the interface begins to exist.

Summary

Composition is not concerned with pixels.

It is concerned with understanding.

Its responsibility is to determine:

What deserves attention?

Everything after Composition concerns how that decision is communicated.

Everything before Composition concerns why that decision exists.

Composition is therefore the bridge between knowledge and interface.

Review Status

Status

Draft

Next File

08-expressions.md

Expressions

Purpose

If Composition decides:

What deserves attention?

Expressions decide:

How should that understanding be communicated?

Expressions bridge the gap between conceptual understanding and user interface.

They intentionally remain independent from visual implementation.

This distinction is one of the most important architectural concepts within Mosaic.

It allows the platform to evolve its interface without changing the underlying understanding of the user's World.

Definition

Within MDL, an **Expression** is defined as:

The conceptual representation chosen to communicate a piece of information within the current composition.

An Expression is **not**:

- a component
- a widget
- a tile
- HTML
- Flutter
- SwiftUI

Those are implementations.

Expressions exist one layer above presentation.

Why Expressions Exist

Traditional UI systems tightly couple information with interface.

For example:

Progress

↓

Progress Bar

This coupling makes future evolution difficult.

Instead, Mosaic separates the concepts.

Progress

↓

Expression

↓

Presentation

The platform first determines **what** it wishes to communicate.

Only afterwards does it determine **how**.

Information Does Not Choose Its Own Expression

This is an important architectural rule.

Information never decides how it should appear.

Example.

Episode Release

↓

Tomorrow

The information remains constant.

Possible Expressions include:

- Timeline
- Countdown
- Hero Detail
- Notification
- Metadata
- Calendar Entry

The appropriate Expression depends entirely upon the current Composition.

Expressions Are Contextual

Expressions change naturally with Context.

Example.

Current Context:

Watching Episode

Episode Release may become:

Timeline

Current Context:

Browsing Calendar

The same information may become:

Calendar Entry

Current Context:

Series Overview

The same information may become:

Hero Subtitle

Nothing about the underlying information changed.

Only its Expression.

Expressions Preserve Meaning

Although Expressions change...

Meaning should remain stable.

The user should recognise:

"This is still the next episode."

regardless of whether it appears as:

- a Timeline
- a Badge
- Hero metadata
- Calendar event

Expression changes should never alter understanding.

Expressions Are Reusable

Multiple information types may share the same Expression.

Example.

Timeline Expression.

Used for:

- episode releases
- chapter progression
- reading history
- album release chronology

Likewise:

Progress Expression.

Used for:

- television
- books
- music
- games

Expressions describe communication patterns rather than media types.

Expressions Are Independent Of Components

Future MDS specifications may implement an Expression using many different components.

Example.

Timeline Expression

Desktop:

Large Timeline Tile

Television:

Condensed Timeline Shelf

Mobile:

Single Timeline Card

Voice Assistant:

Spoken summary

The Expression remains identical.

Only presentation differs.

Good Examples

Example 01

Information:

Reading Progress

↓

68%

Expression:

Progress

Presentation:

Desktop:

Large Progress Tile

Mobile:

Compact Progress Tile

Example 02

Information:

Next Episode

↓

Tomorrow

Expression:

Timeline

Presentation:

Hero Timeline

or

Sidebar Timeline

or

Notification

The platform chooses.

Not the plugin.

Anti-patterns

Information Returning Components

Episode Release

↓

Timeline Widget

Incorrect.

The information has become coupled to presentation.

Plugins Choosing Expressions

Plugin

↓

Hero

Incorrect.

Plugins should contribute information.

The Composition Engine determines Expression.

Components Becoming Concepts

Progress Bar

treated as the architectural concept.

Incorrect.

Progress is the concept.

The Progress Bar is merely one implementation.

Expression Selection

The selection of an Expression depends upon several inputs.

Information

↓

Relationships

↓

Current Context

↓

Composition

↓

Expression

The process intentionally delays presentation decisions until sufficient understanding exists.

Relationship To Tiles

A common misconception is that Expressions are Tiles.

They are not.

Instead:

Expression

↓

Rendered

↓

Tile

Example.

Timeline

↓

Timeline Tile

or

Timeline

↓

Hero Detail

or

Timeline

↓

Notification

The Expression remains identical.

Presentation changes.

Future Runtime

Long-term, the Composition Engine should determine Expressions dynamically.

Future inputs may include:

- device class
- accessibility preferences
- available space
- interaction history
- current Context

The user should experience consistent understanding even as the visual presentation adapts.

Conceptual Model

[mermaid]
flowchart TD

```
Information --> Relationships
Relationships --> Composition
Composition --> Expression
Expression --> Presentation
```

Expressions intentionally separate:

Understanding

from

Rendering.

Design Consequences

Introducing Expressions produces several long-term advantages.

The platform gains:

- reusable communication patterns
- device independence
- plugin consistency
- presentation flexibility
- future-proof architecture

Most importantly...

Information becomes independent from interface.

Summary

Expressions answer one question.

How should this understanding be communicated?

They deliberately sit between Composition and Presentation.

This separation allows Mosaic to evolve visually without redefining the conceptual model established by MDL.

Expressions therefore represent the final conceptual layer before interface begins.

Review Status

Status

Draft

Next File

09-presentation.md

Presentation

Purpose

Presentation is the final stage of the Mosaic Mental Model.

Everything preceding this chapter has deliberately avoided discussing user interface.

Only now, after the platform understands:

- the World
- the current Focus
- the current Context
- Information
- Relationships
- Composition
- Expressions

does interface become relevant.

This ordering is intentional.

Presentation is the consequence of understanding.

It is never the source of understanding.

Definition

Within MDL, **Presentation** is defined as:

The physical manifestation of an Expression using the capabilities of a specific client platform.

Presentation is where concepts finally become interface.

Examples include:

- Tiles
- Typography
- Materials
- Motion
- Colour
- Icons
- Layout

Presentation answers one question.

How should this Expression appear on this device?

Presentation Is Disposable

Unlike the concepts introduced earlier within MDL, Presentation is expected to evolve continuously.

Examples include:

- new Material Systems
- new operating systems
- new rendering technologies
- new interaction devices
- new accessibility capabilities

The Presentation layer should absorb these changes without requiring modifications to:

- World
- Focus
- Context
- Information
- Relationships

This separation is one of the primary architectural objectives of the Mosaic Design Language.

Presentation Is Platform Specific

The same Expression may be presented differently depending upon device capabilities.

Example.

Expression:

Timeline

Desktop.

Timeline Tile

↓

Five upcoming episodes

↓

Artwork

Television.

Timeline Shelf

↓

Two upcoming episodes

Mobile.

Compact Timeline

↓

Single upcoming episode

Voice Assistant.

"The next episode airs tomorrow."

The platform changes.

Understanding remains identical.

Presentation Is Not The Product

One of the easiest mistakes contributors can make is confusing interface with experience.

The interface is only one possible representation of the user's World.

Future platforms may present the same World using:

- augmented reality
- virtual reality
- voice interfaces
- automotive displays
- wearable devices

The Mental Model should remain unchanged.

Presentation should adapt.

Presentation Is Responsible For

Presentation determines:

- visual hierarchy
- typography
- materials
- spacing
- animation
- layout
- responsiveness
- interaction affordances

Presentation intentionally does **not** determine:

- meaning
- importance
- relevance
- relationships

- user intent

Those decisions have already been made earlier in the pipeline.

The Tile

Presentation introduces one important visual primitive.

The Tile.

A Tile is **not** the fundamental unit of Mosaic.

The Tile is merely one presentation mechanism.

Conceptually:

Information

↓

Relationships

↓

Composition

↓

Expression

↓

Tile

The Tile exists because it is an effective way to communicate information across:

- desktop
- mobile
- television
- tablet

Future presentation systems may introduce additional visual primitives without changing the Mental Model.

Materials

Presentation is responsible for choosing materials.

Examples include:

- Canvas
- Acrylic
- Hero
- Overlay

Materials communicate:

- depth

- hierarchy
- separation

Materials should never redefine meaning.

That responsibility belongs to Composition.

Motion

Presentation owns animation.

It does not own behaviour.

Behaviour is defined by:

MDL-004

Presentation merely communicates that behaviour through:

- duration
- easing
- movement
- emphasis

Presentation should therefore never invent interaction.

It should communicate interaction.

Brand

Presentation communicates brand identity.

Brand identity should remain:

- recognisable
- restrained
- supportive

The interface should never overpower entertainment artwork.

Future MDS specifications define:

- brand colour
- atmosphere
- runtime palette
- materials

Presentation applies those systems consistently.

Accessibility

Presentation is responsible for adapting experiences to user needs.

Examples include:

- reduced motion
- high contrast
- larger text
- alternative navigation
- assistive technologies

Accessibility should not alter the underlying Mental Model.

Only its presentation.

The user should experience the same understanding regardless of presentation differences.

Anti-patterns

Presentation Defining Meaning

Example.

Red

↓

Important

Meaning should never depend solely upon colour.

Meaning should already exist before presentation.

Components Becoming Concepts

Treating:

Timeline Tile

as the architectural concept.

Incorrect.

Timeline is the concept.

The Tile is simply one presentation.

Device-Specific Thinking

Designing concepts exclusively around:

- HTML
- Flutter

- SwiftUI

The Mental Model becomes coupled to implementation.

Future evolution becomes unnecessarily difficult.

The Complete Mental Model

The complete conceptual pipeline established by MDL-003 is therefore:

```
[mermaid]
flowchart TD
```

```
World
World --> Focus
Focus --> Context
Context --> Information
Information --> Relationships
Relationships --> Composition
Composition --> Expression
Expression --> Presentation
```

Notice that Presentation appears last.

This ordering is intentional.

Everything before Presentation is independent from interface.

Everything after Presentation belongs to implementation.

Relationship To MDS

The Mental Model ends here.

The next responsibility belongs to the Mosaic Design System.

MDS answers questions such as:

- Which material?
- Which typography?
- Which spacing?
- Which motion?
- Which colour?
- Which component?

MDL intentionally avoids those concerns.

MDL explains:

What should be communicated.

MDS explains:

How it should look.

Summary

Presentation is the physical expression of the user's World.

It should faithfully communicate:

- understanding
- relationships
- hierarchy
- context

without redefining them.

Presentation may evolve continuously.

The Mental Model should remain stable.

This separation allows Mosaic to remain recognisably Mosaic regardless of future platforms or technologies.

Review Status

Status

Draft

Next File

10-user-vs-system-model.md

User Model vs System Model

Purpose

Every successful software platform maintains two separate models.

The first describes how the software actually works.

The second describes how users believe it works.

These models should never be identical.

Instead, the implementation should exist to support the user's understanding while remaining largely invisible.

This chapter formally separates these two models within Mosaic.

Two Models

Every Mosaic implementation contains:

System Model

↓

Mental Translation

↓

User Model

The System Model belongs to engineering.

The User Model belongs to experience.

MDL intentionally defines the User Model.

Engineering remains free to optimise the System Model provided the User Model remains stable.

The System Model

The System Model describes technical reality.

Examples include:

- SQLite
- DuckDB
- GraphQL
- HTTP
- WebSockets
- Plugins
- Authentication

- Caching
- Event Streams
- Composition Engine
- Rendering Pipeline

These concepts are important.

They are simply not important to the user.

The User Model

The User Model describes conceptual reality.

Examples include:

- My World
- My Current Focus
- Continue Watching
- Reading Progress
- Next Episode
- Related Books
- My Collection

These concepts explain the platform without exposing implementation.

Users should never need to translate technical architecture into entertainment concepts.

Mosaic performs that translation for them.

Why The Models Must Differ

Consider the following engineering implementation.

SQLite

↓

DuckDB

↓

Metadata Providers

↓

Relationship Graph

↓

Composition Engine

↓

Renderer

This architecture may be elegant.

It is also meaningless to the majority of users.

The equivalent User Model is:

My World

↓

What I'm Enjoying

↓

What Helps Me Continue

The second model is dramatically simpler.

It therefore becomes the model users naturally adopt.

The Translation Layer

One of Mosaic's primary responsibilities is translating the System Model into the User Model.

```
[mermaid]
flowchart LR
    System["System Model"]
    Translation["Mosaic"]
    User["User Model"]
    System --> Translation
    Translation --> User
```

Users should never experience the System Model directly.

The quality of Mosaic depends upon how effectively this translation occurs.

The Cost Of Leaking Implementation

Implementation details occasionally leak into user experience.

Examples include:

Refresh Metadata

Clear Cache

Plugin Failed

Provider Offline

Mapping Conflict

Sync Database

These concepts describe implementation.

Not entertainment.

Whenever possible they should be translated into language users naturally understand.

Example.

Instead of:

Metadata Refresh Failed

Prefer:

We couldn't update information for this series.

Your existing information is still available.

The implementation remains accurate.

The experience becomes significantly calmer.

Administration

Administration intentionally occupies a unique position.

Administrators occasionally require access to parts of the System Model.

Examples include:

- storage
- plugins
- providers
- users
- diagnostics

Even here, Mosaic should progressively reveal technical detail.

Family members sharing a server should not immediately encounter engineering terminology.

Administrators should be able to move from:

Simple

↓

Advanced

↓

Diagnostic

rather than beginning with raw implementation.

Extensions

Plugins contribute primarily to the System Model.

They provide:

- information
- relationships
- capabilities

They should not redefine the User Model.

Regardless of how many plugins are installed, users should continue experiencing:

- one World
- one Focus
- one Companion
- one Composition model

The platform is responsible for preserving conceptual consistency.

Failure

Failures should be translated through the User Model whenever practical.

System Model.

AniList API Timeout

User Model.

Episode information couldn't be updated.

We'll try again automatically.

You can continue watching normally.

The user understands the consequence.

They do not need to understand the implementation.

Engineering Freedom

Separating these models gives engineering significant freedom.

The following may change without affecting the User Model:

- databases
- APIs
- rendering engines
- frontend frameworks
- plugin architecture
- networking
- storage

Provided the conceptual experience remains stable.

This separation is one of the primary long-term objectives of MDL.

Anti-patterns

Implementation Terminology

DuckDB

SQLite

Provider

GraphQL

HTTP

appearing throughout the user experience.

Plugin Personalities

Plugins introducing competing terminology.

Library

↓

Collection

↓

Shelf

↓

Archive

for the same concept.

The User Model becomes fragmented.

Engineering Navigation

Users navigating implementation rather than entertainment.

Example.

Metadata

↓

Providers

↓

Mappings

instead of

Series

↓

Characters

↓

The implementation has become visible.

User Confidence

The User Model should continuously reinforce confidence.

Users should increasingly feel:

"I know how Mosaic works."

without actually understanding:

- composition engines
- relationship graphs
- runtime systems
- metadata providers

Confidence comes from conceptual consistency.

Not technical knowledge.

Relationship To Future Specifications

Future specifications should continue strengthening this separation.

Examples include:

- Composition Engine
- Runtime Atmosphere
- Extension SDK
- GraphQL UI
- Information Model

These systems should become increasingly sophisticated internally while making the user experience progressively simpler.

Summary

The Mental Model belongs to users.

The System Model belongs to engineers.

Mosaic succeeds when users never need to think about the System Model in order to enjoy their entertainment.

Every future engineering decision should therefore ask:

Does this strengthen the User Model...

or

Does it leak the System Model?

Only the first outcome aligns with the Mosaic Design Language.

Review Status

Status

Draft

Next File

11-governance.md

Mental Model Governance

Purpose

The Mental Model is one of the most stable architectural artefacts within the Mosaic Design Language.

Unlike implementation, which evolves continuously, the Mental Model should change only when there is compelling evidence that it no longer represents the experience Mosaic intends to create.

This chapter defines how the Mental Model is maintained throughout the lifetime of the platform.

Governance Philosophy

The Mental Model is not owned by engineering.

Nor is it owned by design.

It belongs to the product.

Engineering implements it.

Design communicates it.

Product protects it.

This shared ownership ensures the conceptual model remains stable even as implementation evolves.

Conceptual Stability

Every concept introduced within MDL-003 should have an expected lifespan measured in years.

Concept	Expected Lifetime
World	Decades
Focus	Years
Context	Years
Information	Years
Relationships	Years
Composition	Years
Expressions	Years
Presentation	Evolves continuously

The lower layers of the architecture should change significantly more frequently than the higher layers.

This separation allows Mosaic to innovate without forcing contributors or users to relearn the product.

What Requires Review

The following changes require formal Mental Model review.

- Introducing a new conceptual object.
- Removing an existing conceptual object.
- Changing the meaning of an existing concept.
- Introducing alternative conceptual hierarchies.
- Altering relationships between concepts.
- Introducing new architectural abstractions visible to contributors.

These changes affect every downstream specification.

They should never be treated as ordinary implementation work.

What Does Not Require Review

The following changes do **not** normally require Mental Model review.

- New UI components.
- Material changes.
- Typography updates.
- Motion tuning.
- Responsive layouts.
- Platform-specific implementation.
- Rendering optimisations.

These belong to MDS rather than MDL.

The Mental Model Test

Before introducing any new concept contributors should answer the following questions.

Question One

Does this represent something that genuinely exists within the user's entertainment world?

Question Two

Can this concept already be represented by an existing MDL concept?

Question Three

Would users naturally describe their experience using this concept?

Question Four

Will this concept still make sense if Mosaic is rebuilt in ten years?

Question Five

Does introducing this concept simplify the overall model...

...or merely relocate complexity?

Only concepts satisfying all five questions should normally become part of MDL.

Concept Growth

The preferred evolution of the Mental Model is refinement rather than expansion.

Prefer:

Existing Concept

↓

Refinement

instead of:

Existing Concept

↓

New Concept

↓

Exception

↓

Special Case

Every additional concept permanently increases contributor cognitive load.

The Mental Model should therefore remain intentionally small.

Relationship To Engineering

Engineering should implement the Mental Model.

It should not redefine it.

For example.

Engineering may optimise:

- storage
- transport
- rendering

- caching
- networking

Engineering should not introduce new user-facing concepts purely because they simplify implementation.

Conceptual architecture should drive technical architecture.

Not the reverse.

Plugin Governance

Plugins should extend the World.

They should not redefine it.

Examples.

Good:

Book Plugin

↓

Adds

Reading Progress

Poor:

Book Plugin

↓

Introduces

Reading Workspace

The second example introduces a competing conceptual model.

Plugins enrich existing concepts.

They do not create parallel realities.

Future Evolution

The following concepts are intentionally expected to evolve in later specifications.

- Information Graph
- Relationship Graph
- Composition Solver
- Runtime Expressions
- Adaptive Presentation

However...

These evolutions should strengthen the existing hierarchy rather than replace it.

Signs The Mental Model Requires Revision

Revision should be considered only when multiple indicators exist.

Examples include:

- contributors repeatedly invent competing terminology
- extensions consistently require conceptual exceptions
- multiple specifications describe the same idea differently
- users consistently misunderstand the same concept
- engineering repeatedly introduces workarounds around conceptual boundaries

One isolated implementation problem is insufficient justification.

Repeated conceptual friction is.

Governance Workflow

[mermaid]
flowchart TD

```
Problem
Problem --> ExistingConcepts["Existing Concepts"]
ExistingConcepts["Existing Concepts"] --> Refinement["Refinement?"]
Refinement["Refinement?"] -->|Yes| Update
Refinement["Refinement?"] -->|No| NewConceptReview["New Concept Review"]
NewConceptReview["New Concept Review"] --> ADR
ADR --> MDLUpdate["MDL Update"]
MDLUpdate["MDL Update"] --> DownstreamReview["Downstream Review"]
```

The Mental Model should evolve through refinement wherever possible.

Introducing new concepts should remain rare.

Concept Retirement

Concepts should almost never be removed.

If retirement becomes necessary:

1. Mark the concept as deprecated. 2. Introduce its replacement. 3. Record the migration path. 4. Update all downstream specifications. 5. Preserve historical documentation.

The conceptual history of Mosaic is considered valuable architectural knowledge.

Summary

The Mental Model is one of the longest-lived artefacts within Mosaic.

Its purpose is not merely to explain the platform.

Its purpose is to ensure that every contributor, regardless of discipline, is thinking about the same product in the same way.

Protecting that shared understanding is the primary objective of governance.

Architectural Decisions

ADR	Decision
ADR-034	The Mental Model is considered product architecture rather than implementation architecture.
ADR-035	New concepts require formal conceptual review before entering MDL.
ADR-036	Engineering implementation should adapt to the Mental Model rather than redefining it.
ADR-037	Plugins extend existing concepts rather than introducing competing conceptual models.

Review Status

Status

Draft

Next File

12-adrs.md

Architectural Decision Records

Purpose

Architectural Decision Records (ADRs) preserve the reasoning behind the conceptual architecture established within MDL-003.

Unlike implementation decisions, the concepts defined by the Mental Model are expected to influence every future engineering, design and extension decision made within Mosaic.

These ADRs explain **why** those concepts exist.

Future contributors should be able to understand the reasoning behind the Mental Model without needing access to the original design workshops.

ADR Format

Every ADR within MDL follows the same structure.

ADR Number

Status

Context

Decision

Consequences

Alternatives Considered

Related Specifications

One ADR should document one significant conceptual decision.

ADR-049

Title

Treat The User's World As The Primary Organising Concept

Status

Accepted

Context

Traditional media software organises itself around libraries, pages and folders.

Founder discovery consistently described entertainment as a personal world rather than a collection of files.

Decision

The highest-level concept within Mosaic becomes the **World**.

Every subsequent concept exists within it.

Consequences

Future systems should organise information around the user's World rather than software structures.

Navigation should become movement within a World rather than movement between applications.

ADR-050

Title

Separate Focus From Context

Status

Accepted

Context

Early exploration treated Focus and Context as interchangeable concepts.

Further investigation demonstrated they answer different questions.

Decision

Focus answers:

What currently matters?

Context answers:

Why does it matter right now?

Consequences

Future composition systems can adapt to contextual changes without unnecessarily changing Focus.

ADR-051

Title

Treat Information As The Primary Architectural Unit

Status

Accepted

Context

Traditional extension systems generally contribute interface.

This tightly couples capability to presentation.

Decision

Plugins contribute Information.

The platform determines presentation.

Consequences

Future interfaces become:

- reusable
- device independent
- accessible
- composition driven

The extension ecosystem remains visually coherent.

ADR-052

Title

Relationships Are First-Class Concepts

Status

Accepted

Context

Relationships were initially treated as metadata.

During founder workshops it became clear that relationships are responsible for the majority of meaningful exploration.

Decision

Relationships become first-class conceptual objects.

Metadata becomes one possible Information category.

Consequences

Future composition systems should prioritise meaningful relationships over passive categorisation.

ADR-053

Title

Composition Exists Before Interface

Status

Accepted

Context

Many UI frameworks tightly couple composition and rendering.

This makes adaptation difficult.

Decision

Composition becomes a conceptual layer.

Presentation becomes a separate concern.

Consequences

Future rendering systems remain free to evolve while preserving the same understanding.

ADR-054

Title

Introduce Expressions Between Composition And Presentation

Status

Accepted

Context

Directly mapping Composition to UI components created unnecessary coupling.

A missing abstraction existed between understanding and rendering.

Decision

Introduce the concept of Expressions.

Expressions communicate understanding.

Presentation renders Expressions.

Consequences

The same Expression may be rendered differently on:

- desktop
- mobile
- television
- voice interfaces

without changing the conceptual model.

ADR-055

Title

Presentation Is An Implementation Concern

Status

Accepted

Context

Early discussions frequently described interface concepts before conceptual ones.

This encouraged contributors to think in components rather than understanding.

Decision

Presentation becomes the final stage of the Mental Model.

Everything preceding Presentation is implementation independent.

Consequences

Future client applications remain free to innovate while preserving conceptual consistency.

ADR-056

Title

Separate The User Model From The System Model

Status

Accepted

Context

Engineering architecture naturally exposes concepts such as databases, plugins and APIs.

These concepts unnecessarily complicate the user experience.

Decision

The User Model becomes the primary conceptual model described by MDL.

Engineering architecture should translate itself into that model.

Consequences

Implementation remains flexible.

User understanding remains stable.

ADR-057

Title

Mental Model Before Engineering Architecture

Status

Accepted

Context

Many software projects allow implementation constraints to shape conceptual design.

This often results in user experiences that mirror technical architecture.

Decision

The Mental Model becomes the authoritative conceptual architecture.

Engineering architecture should adapt to it wherever practical.

Consequences

Future implementation decisions become easier because conceptual boundaries already exist.

ADR Relationships

[mermaid]
flowchart TD

ADR049["World"]

ADR050["Focus / Context"]

ADR051["Information"]

ADR052["Relationships"]

ADR053["Composition"]

ADR054["Expressions"]

ADR055["Presentation"]

ADR056["User Model"]

ADR057["Mental Model"]

ADR049 --> ADR050

ADR050 --> ADR051

ADR051 --> ADR052

ADR052 --> ADR053

ADR053 --> ADR054

ADR054 --> ADR055

ADR055 --> ADR056

ADR056 --> ADR057

Each ADR intentionally builds upon the previous one.

Together they define the conceptual architecture of Mosaic.

Future ADRs

Future Mental Model ADRs are expected to cover:

- Information Graph
- Relationship Graph
- Domain Model
- World Persistence
- Context Resolution
- Composition Solver
- Runtime Expression Selection

These concepts intentionally remain outside the scope of MDL-003 Version 0.1.

ADR Governance

Mental Model ADRs should rarely change.

Changes should occur only when:

- conceptual ambiguity exists,
- repeated contributor confusion occurs,
- user research invalidates an assumption,
- architectural evolution requires refinement.

Implementation convenience is **not** sufficient justification for changing the Mental Model.

Summary

The ADRs contained within MDL-003 describe the conceptual decisions that make Mosaic possible.

Future engineering systems should treat these decisions as architectural constraints rather than implementation suggestions.

The conceptual integrity of Mosaic depends upon preserving them.

Review Status

Status

Draft

Next File

13-contributor-guidance.md

Contributor Guidance

Purpose

The purpose of MDL-003 is not simply to document concepts.

Its purpose is to change how contributors think.

A contributor who has fully internalised the Mosaic Mental Model should begin solving user problems from conceptual understanding rather than implementation.

This chapter provides practical guidance for applying the Mental Model throughout everyday engineering and design work.

Think In Concepts

One of the easiest mistakes contributors can make is beginning implementation too early.

Avoid thinking:

I need a card.

↓

I need a widget.

↓

I need a GraphQL type.

Instead think:

What does the user need to understand?

↓

What information exists?

↓

How is it related?

↓

What deserves attention?

↓

How should it be expressed?

Implementation naturally follows.

Design From The Top

Contributors should approach every feature using the same conceptual pipeline.

World

↓

Focus



Context



Information



Relationships



Composition



Expressions



Presentation



Implementation

Skipping layers almost always produces poorer experiences.

Build Understanding

Before writing code, contributors should be capable of describing the proposed feature without mentioning:

- components
- pages
- APIs
- GraphQL
- databases
- plugins

Instead describe:

- World
- Focus
- Context
- Information
- Relationships

If this cannot be done, implementation has begun too early.

Ask Better Questions

Poor design questions include:

Where should this button go?

Which page should this live on?

What widget should display this?

Better questions include:

What information exists?

Why is it important?

What relationship does it reveal?

How does it strengthen the user's World?

The second set of questions naturally produces more coherent solutions.

Information First

Contributors should think in terms of Information.

Not interface.

Poor.

Timeline Widget

Better.

Episode Release

Poor.

Recommendation Card

Better.

Relationship

Poor.

Progress Bar

Better.

Progress

Information survives interface.

Interface rarely survives information.

Relationships Before Navigation

When users need additional understanding...

Ask:

Which relationship should be revealed?

before asking:

Which page should we navigate to?

The first strengthens the Mental Model.

The second strengthens software architecture.

Only one of those belongs to MDL.

Expressions Before Components

Contributors should avoid directly selecting components.

Instead determine the required Expression.

Example.

Information

↓

Relationship

↓

Composition

↓

Timeline Expression

↓

Timeline Tile

The component becomes an implementation detail.

This approach significantly improves future flexibility.

Think Like A Companion

Every implementation should reinforce the Companion metaphor.

Before introducing interface ask:

Would a knowledgeable friend mention this?

If yes...

The information probably belongs.

If not...

Question whether the capability genuinely improves the experience.

Plugins

Plugin authors should never ask:

What interface should I build?

Instead ask:

What new knowledge can I contribute?

Examples.

Anime Plugin.

Episode Release

Characters

Studio

Soundtrack

Book Plugin.

Reading Progress

Author

Series Order

Audiobook

The plugin contributes knowledge.

Mosaic contributes understanding.

The World Is Sacred

Every feature exists inside the user's World.

Features should never fragment that World.

Avoid creating concepts such as:

- separate applications
- isolated dashboards
- disconnected workflows

Instead strengthen the existing World.

Design Smells

The following often indicate the Mental Model is being bypassed.

New Terminology

A proposal invents a completely new concept.

Question whether an existing concept already exists.

New Navigation

A proposal introduces another navigation path.

Question whether an existing relationship could reveal the same understanding.

Duplicate Information

The same knowledge appears in multiple places for different reasons.

Question whether composition should determine emphasis instead.

Plugin UI

An extension attempts to create custom interface.

Question whether the extension should instead contribute Information.

Review Questions

Before implementation ask:

- What concept am I implementing?
- Where does it exist within the Mental Model?
- Does this strengthen the user's World?
- Is this Information or interface?
- Does this introduce unnecessary concepts?
- Could another contributor explain this feature using only MDL terminology?

If the answers remain unclear...

Return to the Mental Model.

Mental Model Checklist

Every substantial proposal should satisfy the following.

- ☐ The user's World remains coherent.
- ☐ Focus remains understandable.
- ☐ Context determines relevance.
- ☐ Information remains independent from presentation.
- ☐ Relationships strengthen understanding.
- ☐ Composition determines emphasis.

- [] Expressions remain reusable.
- [] Presentation remains replaceable.

If any item fails, reconsider the design before implementation.

Final Guidance

The Mental Model is not documentation.

It is the conceptual architecture of Mosaic.

Every feature should leave the Mental Model simpler...

...never more complicated.

If contributors consistently think using the concepts introduced by MDL-003, implementation across every client, plugin and platform will naturally begin to feel like one coherent product.

That is the ultimate purpose of the Mental Model.

Review Status

Status

Draft

Next File

glossary.md

Glossary

Purpose

This glossary defines the conceptual terminology introduced by **MDL-003 — Mental Model**.

Unlike implementation terminology, these definitions describe concepts that should remain stable regardless of:

- programming language
- frontend framework
- storage engine
- rendering technology
- client platform

Every future MDL and MDS specification should use these definitions consistently.

Terminology is considered part of the architecture of the Mosaic Design Language.

C

Composition

The intentional organisation of Information that communicates the user's current World with the least possible cognitive effort.

Composition determines:

- hierarchy
- emphasis
- grouping
- importance

Composition does **not** determine presentation.

Context

The temporary circumstances surrounding the user's current Focus.

Context answers:

Why does this matter now?

Context determines relevance.

Context changes continuously.

D

Domain

A major category of entertainment existing within a World.

Examples include:

- Television
- Anime
- Movies
- Books
- Music

Domains organise entertainment.

They do not fragment the World.

E

Expression

The conceptual communication pattern selected by the Composition.

Expressions are independent from components.

Examples include:

- Timeline
- Progress
- Hero Detail
- Metadata
- Notification

Expressions are later rendered through Presentation.

F

Focus

The entertainment experience currently deserving the user's primary attention.

Focus answers:

What currently matters?

Every World normally possesses one primary Focus.

I

Information

A meaningful piece of knowledge describing the user's entertainment world.

Examples include:

- progress
- release date
- runtime
- author
- cast
- rating

Information exists independently from interface.

M

Mental Model

The conceptual explanation of how Mosaic understands the user's entertainment world.

The Mental Model intentionally hides implementation architecture from users.

P

Presentation

The physical rendering of an Expression using the capabilities of a specific platform.

Presentation includes:

- Tiles
- Materials
- Typography
- Motion
- Layout

Presentation should never redefine meaning.

R

Relationship

A meaningful connection between two or more pieces of Information.

Relationships transform information into understanding.

Examples include:

- adaptation
- sequel
- soundtrack
- author
- actor
- franchise

W

World

The highest-level concept within Mosaic.

A World represents the user's complete entertainment environment.

Everything else exists inside a World.

A World is not:

- a homepage
- a library
- a dashboard

It is the conceptual environment experienced by the user.

Conceptual Hierarchy

The Mental Model is intentionally hierarchical.

[text]
world

↓

Focus

↓

Context

↓

Information

↓

Relationships

↓

Composition

↓

Expressions

↓

Presentation

Every concept builds upon the one before it.

Terminology Rules

Future contributors should:

- reuse existing concepts before introducing new ones.
- avoid implementation terminology within MDL.
- preserve conceptual consistency across all specifications.
- favour user language over engineering language.

Concepts should only be renamed through formal design review.

Review Status

Status

Draft

Next File

references.md

References

Purpose

This document records the references and architectural influences that informed **MDL-003 — Mental Model**.

Unlike implementation specifications, these references exist to explain conceptual thinking rather than prescribe implementation.

MDL intentionally synthesises ideas from:

- Human-Centred Design
- Cognitive Psychology
- Information Architecture
- Systems Thinking
- Knowledge Representation
- Software Architecture

The Mental Model itself remains an original architectural model created specifically for Mosaic.

Reading Order

Future contributors should approach references in the following order.

1. MDL Specifications 2. Product Design 3. Human Factors 4. Information Architecture 5. Systems Architecture 6. Engineering References

The Mosaic Design Language always remains the authoritative source.

Internal References

MDL-001 — Vision

Defines:

- Why Mosaic exists
- Product philosophy
- Product beliefs
- Companion metaphor

MDL-003 should always be interpreted as an implementation-independent extension of the Vision.

MDL-002 — Principles

Defines:

- Decision making
- Design principles
- Governance
- Design authority

Every concept introduced by MDL-003 should reinforce one or more MDL-002 principles.

Future Specifications

The following specifications build directly upon the Mental Model.

- MDL-004 Interaction Model
- MDL-005 Composition Model

These specifications intentionally assume the concepts introduced by MDL-003.

They should extend them rather than redefine them.

Future MDS Specifications

The following MDS specifications are expected to implement the Mental Model.

- MDS-001 Design Token Architecture
- MDS-003 Composition Engine
- MDS-007 Tile Framework
- MDS-008 Component Library
- MDS-011 Extension Design Specification

The Mental Model defines:

What exists.

MDS defines:

How those concepts become interface.

Human-Centred Design

Mental Models

Referenced throughout this specification.

Key concepts include:

- conceptual consistency
- user understanding
- predictable behaviour
- reducing cognitive effort

MDL extends these ideas by introducing a domain-specific conceptual architecture centred around entertainment rather than productivity software.

Cognitive Load

Referenced throughout:

- World
- Focus
- Context
- Composition

Reducing unnecessary interpretation is considered a primary optimisation target.

Future Composition specifications are expected to expand upon these ideas.

Information Architecture

The Mental Model intentionally draws inspiration from information architecture rather than traditional interface design.

Primary influences include:

- information before interface
- relationships before navigation
- concepts before components
- understanding before presentation

MDL intentionally extends these concepts into runtime composition.

Knowledge Representation

MDL increasingly treats entertainment as structured knowledge rather than interface objects.

Key influences include:

- entities
- relationships
- semantic meaning
- graph thinking

These ideas are expected to mature significantly within future specifications.

MDL intentionally avoids prescribing implementation technologies for these concepts.

Systems Thinking

The Mental Model intentionally adopts systems thinking rather than feature thinking.

Examples include:

- World instead of Homepage
- Composition instead of Layout
- Information instead of Widgets
- Relationships instead of Navigation

This approach encourages reusable conceptual systems rather than isolated interface features.

Software Architecture

Although MDL intentionally avoids implementation details, the following architectural disciplines influenced its structure.

- Domain-Driven Design
- Event-Driven Systems
- Information Architecture
- Separation of Concerns
- Architecture Decision Records

These references informed documentation structure rather than product philosophy.

Design Systems

The following design systems influenced the organisation of MDL.

Apple Human Interface Guidelines

Influenced:

- philosophy-first thinking
- experience-first design
- restraint

Not adopted:

- visual language
- interaction model

Material Design

Influenced:

- systematic documentation
- design token hierarchy

Not adopted:

- material metaphor
- interaction philosophy

Fluent Design

Influenced:

- adaptive thinking
- cross-device consistency

Not adopted:

- visual identity

Mosaic-Specific Influences

The Mental Model emerged directly from founder discovery workshops.

Major conceptual discoveries included:

- World
- Focus
- Context
- Information
- Relationships
- Composition
- Expressions

These concepts intentionally replaced earlier terminology centred around:

- pages
- widgets
- dashboards
- cards

This evolution represents one of the defining characteristics of the Mosaic Design Language.

Normative References

The following specifications should be considered required reading before contributing to MDL-003.

- MDL-001 Vision

- MDL-002 Principles

Future specifications should reference MDL-003 rather than redefining its concepts.

Informative References

Future contributors may also wish to review:

- MDS-003 Composition Engine
- MDS-007 Tile Framework
- MDS-011 Extension Design Specification

These documents provide implementation guidance for the conceptual architecture established here.

Living Document

This reference list should remain concise.

References should only be added when they have materially influenced:

- conceptual architecture
- terminology
- governance
- design reasoning

The purpose of this document is to preserve architectural context rather than create an exhaustive bibliography.

Completion

This concludes **MDL-003 — Mental Model**.

The next specification in the Mosaic Design Language is:

MDL-004 — Interaction Model

Where MDL-003 defines **how Mosaic understands the world**, MDL-004 defines **how that world behaves over time**.

It formalises concepts including:

- continuity
- transitions
- adaptive behaviour
- composition evolution
- movement
- temporal interaction

MDL-004 transforms understanding into behaviour.